

VNUHCM - UNIVERSITY OF SCIENCE FACULTY OF INFORMATION TECHNOLOGY



CS486 COURSE REPORT

Campus Space Management System

Members of group 05:	MSSV:
Phạm Hữu Nam	24125015
Cao Quảng Hưng	24125010
Nguyễn Hữu Phước	24125018
Trần Đình Quốc Thắng	24125079

TP. Hồ Chí Minh, 06/2026

Mục lục

1	System Overview & LLM Selection	4
1.1	Problem Statement	4
1.2	LLM Models Utilized	4
2	Agent Architecture & Directory Structure	4
2.1	Repository Organization	4
2.2	Layered Documentation Strategy	5
2.3	Agent Workflow	5
2.4	Knowledge Propagation Across Tasks	6
3	Evaluation & Agent Improvement Process	8
3.1	Rubric-Based Evaluation Criteria	8
3.2	Agent Improvement Techniques	9
3.2.1	Task-Scoped Progressive Context Disclosure	9
3.2.2	Command and Skill Metadata with YAML Front Matter	9
3.2.3	Registry-Grounded Source-of-Truth Control	10
3.2.4	Requirement-to-Artifact Traceability Control	10
4	Task-by-Task Evaluation Log	11
4.1	Task 01: Business Requirement Analysis	11
4.1.1	Database Objective	11
4.1.2	Controlled Agent Generation	11
4.1.3	Evaluation Criteria	11
4.1.4	Output Summary	11
4.1.5	Database Design Rationale	11
4.1.6	Issues and Improvements	12
4.1.7	Evaluation Result	12
4.2	Task 02: Conceptual Database Design	13
4.2.1	Database Objective	13
4.2.2	Controlled Agent Generation	13
4.2.3	Evaluation Criteria	13
4.2.4	Output Summary	13
4.2.5	Database Design Rationale	13
4.2.6	Issues and Improvements	14
4.2.7	Evaluation Result	14
4.3	Task 03: Logical Database Design	15
4.3.1	Database Objective	15
4.3.2	Controlled Agent Generation	15
4.3.3	Evaluation Criteria	15
4.3.4	Output Summary	15
4.3.5	Database Design Rationale	16
4.3.6	Issues and Improvements	16
4.3.7	Evaluation Result	17
4.4	Task 04: Database Design Validation	17
4.4.1	Database Objective	17
4.4.2	Controlled Agent Generation	17
4.4.3	Evaluation Criteria	17
4.4.4	Output Summary	18
4.4.5	Database Design Rationale	18

4.4.6	Issues and Improvements	19
4.4.7	Evaluation Result	21
4.5	Task 05: Database Implementation	21
4.5.1	Database Objective	21
4.5.2	Controlled Agent Generation	21
4.5.3	Evaluation Criteria	21
4.5.4	Output Summary	22
4.5.5	Database Design Rationale	22
4.5.6	Issues and Improvements	22
4.5.7	Evaluation Result	24
4.6	Task 06: Sample Data Preparation	24
4.6.1	Database Objective	24
4.6.2	Controlled Agent Generation	24
4.6.3	Evaluation Criteria	24
4.6.4	Output Summary	24
4.6.5	Database Design Rationale	25
4.6.6	Issues and Improvements	25
4.6.7	Evaluation Result	27
4.7	Task 07: Query Design	27
4.7.1	Database Objective	27
4.7.2	Agent Configuration and Improvement Process	27
4.7.3	Per-Student Query Summary	28
4.7.4	Issues and Improvements Summary	29
4.7.5	Evaluation Result	29
5	Conclusion	30

1 System Overview & LLM Selection

1.1 Problem Statement

The School of Computer Science manages a wide range of shared physical spaces, including classrooms, computer laboratories, project laboratories, meeting rooms, auditoriums, and student workspaces. These spaces are used for teaching activities, seminars, examinations, workshops, research, student projects, and academic events. Because many users need access to the same spaces, the School must ensure that room usage is organized, fair, and efficient.

At present, the booking process is handled manually. Lecturers, teaching assistants, students, and staff usually contact the school office or facility staff by email, phone, or in person. Facility staff then check spreadsheets or shared calendars to verify room availability, user eligibility, required facilities, and maintenance status. This manual process becomes increasingly difficult to manage as the number of booking requests grows. It is time-consuming, error-prone, and may lead to conflicting bookings, incomplete records, and inefficient space utilization.

The School therefore needs a database system to support the management of shared campus spaces in a more structured and reliable way. The system should store information about users, bookable spaces, available facilities, booking requests, approval decisions, check-in and completion records, maintenance activities, and historical usage. It must also enforce important business rules, such as preventing overlapping approved bookings for the same space, blocking bookings for spaces that are under maintenance or closed, and recording accountability information for approvals and maintenance actions.

The main goal of the proposed system is to help the School manage shared campus spaces fairly and efficiently, reduce booking conflicts, prevent the use of unavailable spaces, and preserve complete usage history for future reference and decision-making.

1.2 LLM Models Utilized

- **Big Pickle:** Evaluated via the OpenCode Zen platform, this model is specifically optimized for automated coding agents. Featuring an extensive context window of approximately 200k tokens, it serves as an ideal tool for rapid initial prototyping. However, its primary limitation lies in the absence of advanced reasoning features.
- **DeepSeek V4 Flash Free:** Accessed through the OpenCode platform, this model was deployed to facilitate technical drafting, content refinement, and structural editing (e.g., database design documentation). It demonstrates high efficiency in generating structured explanations and concise academic prose, though it necessitates human oversight to ensure rigorous factual accuracy and alignment with project specifications.

2 Agent Architecture & Directory Structure

2.1 Repository Organization

The repository is separated into requirement sources, reusable project knowledge, agent memory, generated deliverables, execution logs, and OpenCode configuration. Each directory has a specific role, which prevents the agent from mixing source material with final outputs or temporary evaluation artifacts. Table 1 maps the major paths to their responsibilities.

Path	Responsibility
README.md	Provides setup instructions, run guidance, and the list of required outputs.
AGENTS.md	Defines agent rules and project-specific operating instructions.
CS486_Project.pdf	Stores the original assignment brief used as the external project reference.
req/	Contains the source business requirements, especially <code>business-requirement.md</code> .
docs/	Acts as the project knowledge base and workflow constraints, including the overview, tech stack, entity registry, schema registry, and design decisions.
docs/templates/	Provides templates and reading or writing guidance for registry-style documentation.
memory/	Stores persistent state such as <code>MEMORY.md</code> , <code>ActiveContext.md</code> , and <code>Progress.md</code> .
outputs/	Contains final deliverables for Tasks 01–07, from requirement analysis to business query design.
logs/trajectory/	Stores task execution traces, grouped by task number.
logs/eval/	Stores output evaluation records and future pipeline-level evaluation logs.
.opencode/commands/	Defines slash commands for running the full database design pipeline, generating individual tasks, and evaluating results.
.opencode/skills/	Stores the main database design pipeline skill, task-specific skills, and the evaluation framework.

Bảng 1: Major repository paths and their responsibilities

2.2 Layered Documentation Strategy

The project adopts an AI-first documentation strategy based on the principle of progressive disclosure [2]. Rather than loading the entire repository into the model context at the beginning of each session, information is organized into multiple layers that are accessed only when relevant to the current task. This design reduces context overload, improves retrieval efficiency, and ensures that stable project knowledge is separated from task-specific procedures and execution history. Table 2 summarizes the four documentation layers used by the agent.

Layer	Primary files	When the agent reads them	Purpose
1	AGENTS.md	At the start of the session	Defines global rules, editing constraints, and workflow expectations.
2	docs/README.md, docs/*.md, docs/templates/	On demand, following the documented reading order	Stores domain knowledge, naming conventions, design decisions, entity definitions, schema definitions, and reusable templates.
3	.opencode/commands/, .opencode/skills/	When a command or task workflow is invoked	Supplies command entry points, pipeline procedures, task rules, and evaluation guidance.
4	memory/*.md	During and after task execution	Preserves active context, progress, corrections.

Bảng 2: Progressive documentation layers used by the agent workflow

2.3 Agent Workflow

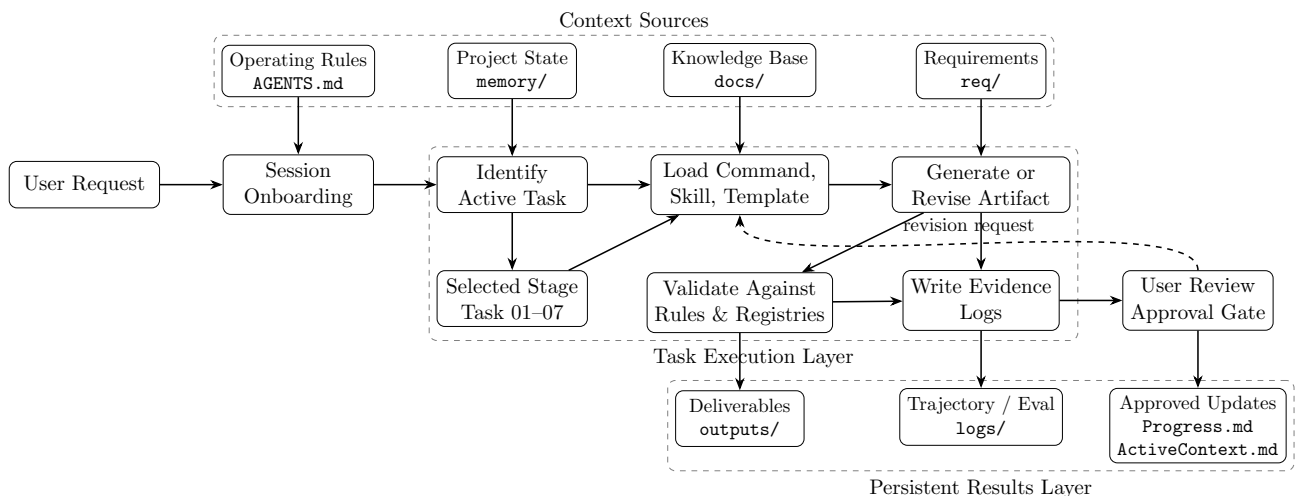
The agent architecture is implemented as a controlled database-design pipeline rather than a single free-form prompting process. When a user invokes a task, the request is first routed to

an OpenCode command, which acts as the workflow entry point. The command identifies the requested stage of the database design lifecycle and loads the corresponding task-specific skill. Examples include business requirement analysis, conceptual database design, logical schema design, database validation, SQL implementation, sample data generation, query design, and evaluation.

Unlike a traditional prompt-based workflow, the system does not rely solely on conversation history to preserve context. Instead, important design decisions are externalized into persistent knowledge artifacts. During execution, the selected skill consults the project knowledge base, reusable templates, and previously generated registries before producing an output. As a result, information discovered in earlier tasks can be reused consistently throughout the pipeline without requiring the model to rediscover the same knowledge multiple times.

The workflow follows a clear separation of concerns. OpenCode commands define task entry points, skills define generation procedures and evaluation rules, the `docs/` directory serves as the authoritative knowledge base, the `memory/` directory maintains persistent project state, the `outputs/` directory stores final deliverables, and the `logs/` directory records execution traces and evaluation evidence. This modular organization improves reproducibility, simplifies auditing, and reduces unintended coupling between different stages of the workflow.

The architecture also incorporates a feedback mechanism. After each task is completed, trajectory logs and evaluation reports are generated and reviewed. Important corrections, unresolved issues, and validated design decisions can then be propagated back into memory and project documentation. Consequently, future tasks benefit from accumulated project knowledge while remaining independent of the original conversation context. Figure 1 illustrates the overall workflow.



Hình 1: Agent Workflow

2.4 Knowledge Propagation Across Tasks

A key design principle of the pipeline is knowledge propagation. Rather than treating each task as an isolated generation step, the system stores important outputs as reusable artifacts that become inputs for subsequent tasks. This approach improves consistency across the database design lifecycle and reduces the risk of contradictory decisions between stages. Knowledge propagation of these artifacts are detailed in Table 3.

Task	Main input(s)	Main output(s)	Role in the pipeline
01	req/business-requirement.md	outputs/01-business-req-analysis-G05.md docs/entity-registry.md	Extracts the business purpose, actors, business rules, candidate entities, attributes, and relationships from the source requirements.
02	docs/entity-registry.md outputs/01-business-req-analysis-G05.md	outputs/02-erd-design-G05.md updated entity registry	Converts conceptual entities into an ERD, confirms relationships, cardinalities, participation constraints, and junction entities.
03	docs/entity-registry.md outputs/02-erd-design-G05.md	outputs/03-logical-design-G05.md docs/schema-registry.md	Translates the ERD into a normalized relational schema, finalizes names and data types, and populates the schema registry.
04	docs/entity-registry.md docs/schema-registry.md outputs/03-logical-design-G05.md	outputs/04-design-validation-G05.md schema-freeze decision	Validates requirement coverage, ERD fidelity, normalization, and schema consistency before freezing the schema.
05	docs/schema-registry.md outputs/04-design-validation-G05.md	outputs/05-db-definition-G05.sql DDL compile logs	Generates SQL Server DDL from the frozen schema, including tables, constraints, indexes, triggers, and validation-supporting database logic.
06	outputs/05-db-definition-G05.sql docs/schema-registry.md docs/entity-registry.md	outputs/06-sample-data-G05.sql logs/execution/task06/ logs/eval/task06/	Produces realistic, idempotent sample data and expected-error tests that prove key business rules against the approved schema.
07	outputs/05-db-definition-G05.sql outputs/06-sample-data-G05.sql outputs/01-business-req-analysis-G05.md req/business-requirement.md	outputs/07-query-design-G05.sql logs/eval/task07/	To produce personalized, meaningful, parameterized, and validated T-SQL queries based on student ideas that directly resolve core operational and analytic questions from role-based perspectives without loss of prior work.

Bảng 3: Task-specific artifacts used across the database design pipeline

3 Evaluation & Agent Improvement Process

3.1 Rubric-Based Evaluation Criteria

The evaluation framework was designed to measure both the quality of the generated database artifacts and the quality of the agent process used to produce them. This separation follows agent-evaluation guidance that recommends assessing not only final answers, but also reasoning traces, tool use, and intermediate actions [3, 4, 5, 6]. Therefore, each task was evaluated with two independent scores: an **artifact quality score** and an **agent process quality score**. The two scores were intentionally not merged, because a task can produce a correct deliverable while still using an incomplete, inefficient, or unsafe workflow.

Artifact quality rubric. Artifact quality was scored from 0 to 5 using a weighted database-design rubric. A score of 0 means the criterion is missing or seriously wrong, while a score of 5 means the output is strong, complete, consistent, and well justified. The weighted artifact score is calculated as the sum of each criterion score multiplied by its weight.

Criterion	Weight	Evaluation focus
Completeness of required outputs	10%	Required files exist and contain all mandatory sections for the active task.
Requirement coverage	20%	Source requirements and business rules are preserved, especially booking conflicts, unavailable-space restrictions, status constraints, and maintenance rules.
Database design correctness	20%	Entities, relationships, cardinalities, tables, keys, constraints, triggers, and indexes are technically reasonable and enforce the intended rules.
Traceability	15%	Requirements can be followed through business rules, entities, relationships, tables, constraints, sample data, and queries.
Assumptions and open questions	10%	Missing or ambiguous information is explicitly recorded instead of being silently invented.
Cross-file consistency	10%	Entity names, table names, attributes, statuses, and constraints remain consistent across outputs, registries, and design decisions.
SQL Server compatibility	10%	SQL artifacts use valid Microsoft SQL Server syntax and provide execution evidence where required.
Query usefulness	5%	Final queries answer realistic operational needs such as booking history, utilization, no-shows, maintenance, and unavailable spaces.

Bảng 4: Artifact quality rubric used for generated database design outputs

Agent process rubric. Process quality was evaluated from the trajectory logs rather than from the final artifact alone. This matches trajectory-based evaluation for multi-step agents, where silent process failures can occur even when the final answer appears plausible [4, 6]. After generating or revising an **outputs/OX-*** artifact, the agent records the plan, actual execution steps, files read and written, deviations, outcome, and self-detected fixes. The six process metrics are scored independently from 0 to 5 and then averaged into a process score.

Metric	Evaluation focus
PlanQuality	Whether the plan is logical, ordered, complete, and suited to the active pipeline task.
PlanAdherence	Whether actual execution followed the plan or justified deviations clearly.
StepEfficiency	Whether the agent avoided redundant reads, writes, retries, and unnecessary context loading.
TaskCompletion	Whether the required output exists and satisfies the task’s success criteria.
ToolCorrectness	Whether the agent read and wrote the expected files while avoiding forbidden paths.
ArgumentCorrectness	Whether commands and file targets used correct paths, group suffixes, and read/write directions.

Bảng 5: Agent process metrics scored from trajectory logs

Evaluation workflow. The recommended workflow is iterative: the agent first generates a task artifact, then writes a trajectory log, then the evaluator runs a task-level evaluation when a quality check is needed. The evaluation report returns an artifact score, a process score, supporting evidence, and the top improvement actions. Those results are then used to revise the output, template, command, or task-specific skill when needed. At larger milestones, a pipeline-level evaluation checks cross-task consistency. This reflects the practice of combining end-to-end artifact evaluation with component-level checks for tool choice, file access, arguments, and side effects [5, 6, 7].

3.2 Agent Improvement Techniques

3.2.1 Task-Scoped Progressive Context Disclosure

The first improvement technique was **task-scoped progressive context disclosure**. Instead of loading the entire repository, the agent followed the documented reading order and opened only the files relevant to the active pipeline stage. The process started with project rules and memory files, then moved to task-specific skills, templates, and the exact upstream artifacts required for the current deliverable.

This technique matched the seven-task pipeline. For Task 01, the authoritative input was `req/business-requirement.md`, so the agent avoided premature schema assumptions. For Tasks 02 and 03, the context expanded to the entity registry, ERD, business-rule analysis, and technical conventions. After schema freeze, Tasks 05 and 06 relied primarily on the locked schema registry, approved validation output, DDL, and task-specific SQL guidance.

This approach aligns with the progressive disclosure principle [9] and improved the agent in three ways: **First**, it reduced context pollution by excluding unrelated files. **Second**, it lowered cross-task drift via controlled inputs. **Third**, it simplified evaluation through verifiable logs.

3.2.2 Command and Skill Metadata with YAML Front Matter

The second improvement technique was the use of **YAML front matter** as a small machine-readable header before longer human-readable instructions. YAML front matter is a common pattern for storing file-level metadata at the beginning of Markdown-like documents [10]. In this project, it appeared in command files, skill files, and trajectory logs.

In `.opencode/commands/`, front matter helped identify command-level properties before the prompt body was read. In `.opencode/skills/`, it exposed `name` and `description` fields

that helped the agent select task-specific behavior. In `logs/trajectory/`, it standardized run metadata such as `task`, `group`, `run_at`, `status`, and `revision_of`. OpenCode's command and skill documentation directly supports this style of command routing and skill discovery [11, 12].

This improved the agent in three ways: **First**, it strengthened command routing through early command distinction. **Second**, it enhanced skill discovery without loading full task files. **Third**, it increased evaluation reliability via stable trajectory identifiers. To prevent metadata drift, schema facts and business rules were strictly maintained in authoritative registries rather than front matter.

3.2.3 Registry-Grounded Source-of-Truth Control

The third improvement technique was **registry-grounded source-of-truth control**. Stable database knowledge was externalized into persistent documentation: `docs/entity-registry.md` for conceptual entities and attributes, `docs/schema-registry.md` for relational tables and constraints, and `docs/design-decisions.md` for rationale and trade-offs.

This structure matched the project pipeline. Task 01 populated the entity registry from the business requirement. Task 02 refined relationships and cardinalities. Task 03 locked attributes and populated the schema registry. Task 04 validated both registries before schema freeze. After that point, Tasks 05–07 had to treat the registries as read-only control documents rather than silently modifying upstream design facts.

This improved the agent in three ways. **First**, it reduced context drift because later tasks did not need to rediscover the entity model from scratch. **Second**, it improved cross-file consistency because ERD, logical schema, DDL, sample data, and future query design could be checked against the same approved registries. **Third**, it introduced change control: if a later task needed to revise a design choice, the reason had to be recorded in `docs/design-decisions.md` rather than hidden inside a generated output.

This approach aligns with both **codified context infrastructure** and **source-of-truth synchronization** in agent systems. In our project, the registries acted as **domain-specific cold memory**, while the design-decision log served as a **governance record**. Instead of allowing duplicated local representations to drift, the agent must consult these canonical registries before producing or validating downstream artifacts. Together, this framework made the workflow more **auditable, reproducible, and resistant to hallucinated schema changes**.

3.2.4 Requirement-to-Artifact Traceability Control

The fourth improvement technique was **requirement-to-artifact traceability control**. Each important requirement needed a visible path through the pipeline: from the original business requirement, to numbered business rules, entities, relationships, logical tables, database constraints, DDL triggers, sample-data tests, and eventually business queries. This made it easier to verify that requirements were preserved instead of being dropped or reinterpreted across tasks.

In this project, traceability appeared as a distributed control mechanism rather than a single perfect matrix. Task 01 established numbered business rules and assumptions. The entity registry stored source references for conceptual entities and attributes. The schema registry mapped business rules to database-level enforcement mechanisms such as constraints, indexes, and triggers. Task 06 then strengthened downstream traceability by labeling expected-error cases with BR numbers and preserving SQL execution evidence.

This improved the agent in three ways. First, it enabled coverage checking by exposing missing requirement links. Second, it facilitated backward debugging by tracing design errors back to source rules. Third, it enhanced auditability by making the requirement's entire route

inspectable. This is consistent with requirements traceability research, which treats trace links as a way to follow requirements forward and backward through development artifacts [13].

4 Task-by-Task Evaluation Log

4.1 Task 01: Business Requirement Analysis

4.1.1 Database Objective

Task 01 analysed the project requirements to establish a complete understanding of the business domain before any design activities.

4.1.2 Controlled Agent Generation

The primary source of information was `req/business-requirement.md`, supported by project documentation defining the business scope, technology context, entities, design decisions, and current working state.

4.1.3 Evaluation Criteria

Task 01 was evaluated using the business-requirement-analysis file-level checks from the project rubric. Because this task defines the database scope for all later design work, the criteria focused on whether the requirement artifact captured the full business domain and made critical rules explicit.

- **Requirement analysis completeness:** stakeholders, business processes, candidate entities, relationships, assumptions, and open questions must be present.
- **Business-critical constraints:** time-overlap prevention and maintenance/status-based booking blocking must be identified as top-priority rules.
- **Traceability evidence:** the artifact must include a section checklist, a business-rule list, and a traceability matrix linking rules back to the requirement source.

4.1.4 Output Summary

The task produced `outputs/01-business-req-analysis-G05.md`. The output summarised the project purpose, six user roles, core domain objects, booking and approval workflows, maintenance handling, 14 business rules, assumptions, open questions, and a requirements traceability matrix linking business rules back to `req/business-requirement.md`.

4.1.5 Database Design Rationale

The conceptual and logical design is established based on four core analytical principles:

- **Entity Identification:** Distinct real-world objects (`users`, `spaces`, `facilities`, `bookings`, `maintenance`, and `departments`) are mapped to independent entities to preserve semantic clarity. Additionally, the many-to-many relationship between spaces and facilities is explicitly resolved using the `space_facilities` junction entity during early analysis to bound the design scope.
- **Business Rule Classification:** Rules are categorized into *explicit* (directly enforceable constraints), *inferred* (requiring domain-owner validation, such as expected participants $\leq$ capacity), and *design conventions* (technical standards like audit timestamps). This prevents downstream tasks from treating all rules as equally mandatory, avoiding over-

constrained schemas or unvalidated assumptions.

- **Decoupling of the Booking Lifecycle:** Booking requests, approval decisions, and session check-ins are analyzed as separate concerns due to differences in timing, actors, and data production. Separating them at the requirements level prevents the necessity of nullable columns (e.g., pending bookings lacking approvers or check-in times) in a monolithic table.
- **Entity Consolidation and Boundary Management:** Incident reporting is consolidated into the **maintenance** entity via the **problem_description** field, as it lacks distinct attributes to justify a standalone schema complexity. Furthermore, unstated assumptions and open questions are explicitly documented to force early validation and prevent scope disputes caused by incomplete requirements.

4.1.6 Issues and Improvements

Task 01 required three evaluation rounds before approval. The initial version captured the core business requirements but lacked several documentation and traceability elements required by the evaluation framework. Table 6 summarises the major issues identified, the corresponding improvements introduced, and their effect on the final artifact.

Issue Found	Improvement Made	Effect
The initial version lacked a requirements traceability matrix.	A 14-row traceability matrix was added, mapping each business rule to source evidence and classifying it as explicit, inferred, or design-convention based.	Improved requirements traceability and source coverage.
Incident reporting was not analysed as a separate concern.	The analysis was expanded to explicitly include incident reporting within the business requirements.	Improved completeness of business process coverage.
Entity relationships did not include explicit cardinality notation.	Entity relationships were revised to include explicit cardinality notation.	Improved clarity of relationship analysis for subsequent design tasks.
Related formatting and task state updates were incomplete.	Formatting and memory updates were completed, and Task 01 was marked approved before proceeding to Task 02.	Improved workflow consistency and task progression.

Bảng 6: Issues identified and improvements made during Task 01 iterations

4.1.7 Evaluation Result

Task 01 was evaluated across three rounds using both artifact-level and process-level criteria. The final artifact score was **4.40/5.00**, while the process score reached **4.50/5.00**. All eight file-level checks passed, including stakeholder coverage, process coverage, constraints, candidate entities, relationships, assumptions, open questions, and traceability.

The iterative refinement primarily strengthened documentation completeness, requirements traceability, and relationship analysis, resulting in an approved business requirements specification that served as the foundation for subsequent design tasks.

4.2 Task 02: Conceptual Database Design

4.2.1 Database Objective

Task 02 transformed the approved business analysis into a conceptual entity-relationship model.

4.2.2 Controlled Agent Generation

The primary source of information was `docs/entity-registry.md`, while `outputs/01-business-req-analysis-G05.md` was consulted only to resolve requirement ambiguities. Memory files and project instructions provided additional task context.

4.2.3 Evaluation Criteria

Task 02 was evaluated using the conceptual-ERD file-level checks from the project rubric. Since this task is a conceptual database-design artifact, the criteria emphasized correct ER modelling rather than physical SQL implementation.

- **Conceptual ERD completeness:** entity and relationship summaries must be present.
- **Cardinality correctness:** the Mermaid `erDiagram` must use correct Crow's Foot cardinality and participation notation.
- **Constraint separation:** logical business constraints must be documented outside the ERD rather than embedded as physical database implementation details.
- **Conceptual purity:** entity blocks must avoid SQL physical types, audit columns, and explicit foreign-key markers.
- **Review evidence:** the artifact must provide Mermaid review, a relationship participation table, and a conceptual-purity note.

4.2.4 Output Summary

The task produced `outputs/02-erd-design-G05.md` containing:

- **9 conceptual entities:** Departments, Users, Spaces, Facilities, Space_Facilities, Bookings, Booking_Approvals, Booking_Sessions, Maintenance.
- **11 relationships (R1–R11)** mapped in Mermaid.js ERD with two diagrams — an overview (nodes only) and a full-detail diagram showing attributes and PK markers.
- **Relationship Participation Summary** — a 5-column table documenting cardinality, Mermaid notation, and total/partial participation justification for every relationship.
- **6 Logical Constraints** — role-enforcement rules for approver, check-in staff, and maintenance assignee; booking non-overlap; space availability; and soft deletion.
- **Pre-Submission Validation Checklist** — 18 items verifying entity coverage, relationship completeness, cardinality correctness, and conceptual-model purity.

4.2.5 Database Design Rationale

The booking lifecycle was split into `bookings`, `booking_approvals`, and `booking_sessions` to avoid NULL sprawl: in a monolithic table, approval and session columns would be empty for every newly submitted booking. The split also models optional lifecycle steps, such as pending requests and no-shows, without placeholder rows, and enforces 1:0..1 cardinality through UNIQUE foreign keys.

The many-to-many relationship between spaces and facilities was resolved using a `space_facilities` junction table with a composite primary key and optional quantity. This keeps the schema normalised and supports queries such as “which spaces have a projector?” without repeating facility columns.

Maintenance assignment is optional because a ticket may be reported before a technician is assigned. The nullable `assigned_staff_id` with ON DELETE SET NULL preserves the maintenance record even if the assigned staff member is later removed.

Building and floor remained free-text attributes on `spaces` rather than separate reference tables, since no requirement demands cross-building reporting that would justify the extra joins.

Role-based constraints, such as who may approve, check in, or handle maintenance, were documented as logical constraints rather than embedded in the ERD because they require cross-table validation against `users.role` and are better enforced as triggers in the logical design phase.

4.2.6 Issues and Improvements

Task 02 required three iterations before reaching a fully compliant conceptual ERD. The first implementation established the overall structure, while later iterations addressed documentation completeness and conceptual-model purity. Table 7 summarises the major issues identified, the corresponding improvements introduced, and their effect on the final artifact.

Issue Found	Improvement Made	Effect
The initial artifact lacked four mandatory sections required by the specification: <i>Relationship Participation Summary, Logical Constraints, Design Decisions, and the Pre-Submission Validation Checklist.</i>	All four missing documentation sections were added during the second iteration, achieving full functional and structural completeness.	Improved compliance with the required artifact structure.
The ERD contained physical database concepts, including SQL datatypes, audit columns, and foreign key markers.	The final iteration enforced strict conceptual purity by replacing physical types with generic types, removing audit columns from all nine entities, and eliminating all foreign key markers.	Produced a fully conceptual ERD aligned with the design rules.
The initial workflow did not verify the generated output against the required skill output specification before completion.	The underlying workflow and <code>SKILL.md</code> rules were revised to enforce the required conceptual-model constraints.	Improved alignment between the generation process and the conceptual design requirements.

Bảng 7: Issues identified and improvements made during Task 02 iterations

4.2.7 Evaluation Result

Task 02 was evaluated across three iterations using both artifact-level and process-level criteria. The final artifact score was **4.75/5.00**, while the process score reached **4.50/5.00**. The final artifact passed the required conceptual-design checks, including entity coverage, relationship participation, logical constraints, design decisions, pre-submission validation, and conceptual purity.

The iterative refinement primarily strengthened documentation completeness, relationship

notation, and ERD purity by removing SQL datatypes, audit columns, and foreign key markers from the conceptual model. The approved conceptual ERD therefore became a clean foundation for the logical database design in Task 03.

4.3 Task 03: Logical Database Design

4.3.1 Database Objective

Task 03 transformed the approved conceptual database design into a complete logical database schema for Microsoft SQL Server.

4.3.2 Controlled Agent Generation

The primary inputs were the business analysis from Task 01, the conceptual ERD from Task 02, together with the entity, technical conventions, design decisions, and the original business requirements. These artifacts provided the approved entities, relationships, SQL Server rules, naming conventions, and unresolved ambiguities used to construct the logical schema.

4.3.3 Evaluation Criteria

Task 03 was evaluated using the logical-schema file-level checks from the project rubric. The criteria focused on whether the conceptual ERD had been translated into a complete, normalized, SQL Server-oriented logical schema with database-level enforcement for critical booking rules.

- **Logical schema completeness:** tables, columns, data types, primary keys, foreign keys, UNIQUE, CHECK, and DEFAULT constraints must be documented.
- **Referential design:** all relationships must include explicit ON DELETE and ON UPDATE actions.
- **Index strategy:** the schema must document indexes that support keys, foreign keys, and important booking queries.
- **Naming conventions:** table, column, key, constraint, index, and trigger names must follow the project conventions.
- **Database-level rule enforcement:** trigger logic must be specified for cross-table rules, especially advanced booking and workflow constraints such as overlap prevention.
- **Design validation evidence:** the artifact must include a table-definition checklist, relationship-to-FK matrix, constraint/index review, ERD-to-logical mapping, justified ERD deviations, and a 1NF/2NF/3NF verdict.

4.3.4 Output Summary

The task produced outputs/03-logical-design-G05.md containing:

- **9 normalized tables** — departments, users, spaces, facilities, `space_facilities`, bookings, `booking_approvals`, `booking_sessions`, and maintenance, each with SQL Server-compatible columns, data types, PKs, FKs, and CHECK constraints.
- **11 relationship mappings** (R1–R11) with per-FK referential integrity rules, including CASCADE, NO ACTION, and SET NULL.
- **30 indexes** covering clustered PKs, UNIQUE business keys, FK-acceleration indexes, a filtered unique index for lightweight overlap pre-checks, and a covering index for interval overlap detection.
- **18 business rules (BR1–BR18)** enforced through CHECK, UNIQUE, FK constraints, and 13 AFTER triggers for overlap prevention, capacity checks, maintenance conflicts, role validation, status sync, space cleanup on cancellation, and audit timestamps.

- **5 resolved ambiguities** from Task 01 covering rejection reason handling, usage policy format, maintenance–booking interaction, no-show detection, and building/floor modelling.
- **4 documented ERD deviations** with justification, including account status enum finalisation, maintenance soft delete, and role triggers upgraded from application to database level.
- **Normalization proof** from 1NF to 3NF, presented table-by-table with evidence.
- **Naming conventions** for tables, columns, PKs, FKs, indexes, constraints, and triggers.

4.3.5 Database Design Rationale

Overlap prevention uses a dual-layer design: a filtered unique index plus a trigger. A single trigger could detect every overlap, but it would fire on every `INSERT` and `UPDATE`, even when no collision is likely. The filtered unique index on (`space_id`, `requested_start_time`) for active booking statuses rejects exact start-time collisions at index level as a cheap pre-check. The trigger then handles interval overlaps, such as 10:00–12:00 against 11:00–13:00, which a unique index cannot express. Together, they give complete coverage while improving performance for the common exact-collision case.

Referential actions use an asymmetric `CASCADE` policy. `booking_approvals` and `booking_sessions` are dependent child tables, so deleting a booking should also delete its approval and session records because those rows have no meaning without the parent booking. In contrast, foreign keys referencing `users` use `NO ACTION` because deleting a user should not automatically remove historical booking or maintenance records. This also avoids SQL Server’s multiple cascade-path restriction, which could occur if requester, approver, or staff references all cascaded through `users`.

Statuses and roles are implemented as `VARCHAR` values with `CHECK` constraints instead of lookup tables. The allowed values, such as user roles, booking statuses, and maintenance statuses, are fixed by the requirements and do not need runtime extension. Normalising them into lookup tables would add joins to common queries without a clear benefit. `CHECK` constraints provide the needed database-level integrity by rejecting invalid values while keeping the schema simple and query execution efficient.

4.3.6 Issues and Improvements

Task 03 required several refinements before reaching the final logical design. The initial version satisfied the overall logical schema requirements, but evaluation identified several issues related to business-rule enforcement, document formatting, and workflow discipline. Table 8 summarises the issues found, the corresponding improvements, and their effect on the final deliverable.

Issue Found	Improvement Made	Effect
BR8 and BR9 were initially treated as application-level rules instead of database-enforced rules.	The logical design separated actual usage tracking into <code>booking_sessions</code> and added trigger-level rules for check-in, completion time, and space condition recording.	Ensured actual time and condition tracking were represented as database-level rules instead of informal application behavior.
One Markdown table contained a column separator mismatch.	The table formatting issue was corrected.	Improved document consistency and formatting correctness.
One revision updated a Task 04 file outside the expected Task 03 write scope.	The logical design revision was updated to v1.2 and the validation report was synchronized accordingly.	Restored consistency between the logical design and validation artifacts while keeping the documented business-rule enforcement aligned.

Bảng 8: Issues identified and improvements made during Task 03 iterations

4.3.7 Evaluation Result

Task 03 achieved a perfect applicable artifact score of **4.75/4.75**, demonstrating complete logical schema coverage, requirement traceability, database correctness, consistency, SQL Server compatibility, and completeness. The process score was **4.67/5.00**. All file-level checks passed, including verification of tables, columns, keys, constraints, indexes, trigger logic, normalization, and naming conventions.

The final logical design fully satisfied the applicable artifact evaluation criteria, while the remaining process deductions reflected workflow issues rather than deficiencies in the logical database schema itself.

4.4 Task 04: Database Design Validation

4.4.1 Database Objective

Task 04 validated the logical database design before schema freeze by checking all approved upstream artifacts for structural consistency, business-rule coverage, normalization, and SQL Server compliance.

4.4.2 Controlled Agent Generation

The validation was performed against the approved business analysis, conceptual design, logical design, registries, design decisions, and technical conventions.

4.4.3 Evaluation Criteria

Task 04 was evaluated using the design-validation file-level checks from the project rubric. As the schema-freeze gate, this task had to verify that the logical database design was complete, consistent, and safe to use as the source for DDL generation.

- **Requirement and rule coverage:** every requirement and business rule must be checked against the logical schema.
- **Relationship translation:** each conceptual relationship must be implemented with the correct foreign key, unique foreign key, or junction-table structure.
- **Participation and nullability:** required and optional participation must match the nullability and referential actions in the logical schema.

- **Normalization and anomaly checks:** the validation must assess 3NF compliance and identify insertion, update, or deletion risks.
- **Gap and risk handling:** the report must include a gap/risk log and fix recommendations for any unresolved design issues.
- **Cross-artifact consistency:** indexing, conceptual-vs-logical deltas, registry synchronization, indexing/partitioning, and naming consistency must be checked.
- **Schema-freeze evidence:** the report must include a validation checklist, discrepancy log, registry lock status, and explicit schema-freeze verdict.

4.4.4 Output Summary

The task produced a comprehensive validation report covering the complete logical database design, as a gate between logical design and physical implementation.

The validation applies five criteria derived from the requirements:

- **Correctly represents the ERD:** Every entity, attribute, and relationship from the ERD must appear in the logical schema with matching structure.
- **Satisfies business rules:** Every business rule must have a corresponding enforcement mechanism, such as `CHECK`, `FK`, `UNIQUE`, trigger, or index.
- **Uses appropriate keys:** Every table must have a primary key; surrogate keys must be justified; business keys must have `UNIQUE` constraints.
- **Uses appropriate relationships:** Cardinalities, participation constraints, and referential integrity actions must match the ERD.
- **Uses appropriate constraints:** `NOT NULL`, `CHECK`, `UNIQUE`, `DEFAULT`, and foreign-key constraints must be complete and correct.
- **Verifies Third Normal Form:** The logical schema must be checked for 3NF compliance, ensuring that tables avoid partial dependencies, transitive dependencies, and unnecessary redundancy.

4.4.5 Database Design Rationale

Correct ERD Representation This criterion checks whether the logical schema faithfully translates the conceptual ERD. The ERD is the conceptual contract between the design team and the School: it defines business entities such as spaces, bookings, facilities, and maintenance records, together with the relationships among them. The logical schema must preserve that contract through tables, columns, primary keys, and foreign keys. If this translation is wrong, the database may compile correctly but fail to support the workflows expected by stakeholders.

Business Rule Satisfaction This criterion verifies that every business rule is enforced at the database level. Correct table structure only ensures that the database can store the required data; it does not prevent invalid data from being stored. Rules such as booking-overlap prevention, unavailable-space blocking, authorization checks, and mandatory rejection reasons require active enforcement through `CHECK`, `FK`, `UNIQUE`, triggers, filtered indexes, or related mechanisms. Database-level enforcement provides defense in depth because it protects integrity even when data enters through scripts, direct SQL access, or future application changes.

Appropriate Key Strategy This criterion checks whether each table has a valid primary-key strategy. Every row must be uniquely identifiable so that foreign keys, updates, deletes, and indexes have stable targets. The validation therefore examines both surrogate keys and business keys. The hybrid design, using small surrogate `INT IDENTITY` primary keys together

with **UNIQUE** constraints on natural business keys, supports efficient joins while still preventing duplicate emails, space codes, department codes, and other real-world identifiers.

Relationship Correctness This criterion verifies that ERD relationships are implemented with correct cardinality, foreign-key placement, uniqueness constraints, and referential actions. Relationship translation is a high-risk step because a misplaced foreign key or missing **UNIQUE** constraint can silently change the meaning of the model. For example, a 1:0..1 approval relationship requires a dependent record and uniqueness on the booking reference; otherwise the schema could allow multiple approvals for one booking. Appropriate **ON DELETE** behavior is also required to preserve historical records while allowing dependent cleanup where justified.

Constraint Completeness This criterion checks whether column domains and required values are properly bounded. Foreign keys only prove that referenced rows exist; they do not prove that values are meaningful. Constraints such as **NOT NULL**, **CHECK**, **UNIQUE**, and **DEFAULT** prevent invalid states such as negative capacity, missing roles, invalid status values, or incomplete booking records. Each missing constraint represents a data-integrity gap that would need to be handled repeatedly in application code and cleanup queries.

Third Normal Form Verification This criterion verifies that the schema satisfies the project requirement for at least Third Normal Form (3NF). Normalization prevents the same fact from being stored in multiple places, which reduces update, insertion, and deletion anomalies. A schema can appear complete and still be structurally weak if it contains partial dependencies, transitive dependencies, or unnecessary redundancy. Therefore, 3NF verification acts as a final structural correctness check before the design is approved for physical implementation.

4.4.6 Issues and Improvements

Task 04 was refined over three validation iterations to improve consistency, database-level enforcement, and cross-file synchronization. Table 9 summarizes the issues identified, the corresponding improvements, and their effects on the final validation report.

Issue Found	Improvement Made	Effect
Missing performance indexes in the schema registry and one index naming discrepancy across registries.	Four missing indexes were added to <code>schema-registry.md</code> , and the inconsistent index name was unified as <code>idx_bookings_time_range</code> .	Improved index completeness and cross-registry consistency.
Three minor constraint gaps remained for facility quantity, task completion time, and actual booking end time.	The gaps were documented and intentionally deferred to the DDL implementation stage in Task 05.	Preserved validation completeness while assigning implementation to the appropriate development stage.
BR8 and BR9 were still treated too much as application-level logic, and the approval/check-in responsibilities were concentrated in <code>bookings</code> .	The validation was updated to support the split design: <code>booking_approvals</code> records approval decisions and rejection reasons, while <code>booking_sessions</code> records check-in, actual time, and condition information. The report also added self-check and idempotency sections.	Strengthened database-level rule enforcement and clarified the separation of booking request, approval, and usage-session responsibilities.
A naming mismatch existed between the logical design and schema registry for the maintenance table.	The schema registry standardized the physical table name as <code>maintenance</code> ; related indexes, foreign keys, and downstream references were re-verified before schema freeze.	Completed cross-file synchronization with the final 9-table schema and removed the previous singular/plural ambiguity.

Bảng 9: Issues identified and improvements made during Task 04 iterations

4.4.7 Evaluation Result

Task 04 achieved a weighted evaluation score of **4.65/5.00**. The validation demonstrated complete requirement coverage by verifying the expanded business-rule set, confirmed the normalized 9-table schema, maintained clear traceability across upstream artifacts, documented assumptions and remaining constraints, verified SQL Server compatibility, improved query usefulness through additional performance indexes and concluded with **Schema Freeze Approved**, allowing downstream DDL generation.

The remaining deductions reflected minor completeness and cross-file naming issues rather than structural deficiencies in the validated database design.

4.5 Task 05: Database Implementation

4.5.1 Database Objective

Task 05 converted the schema-frozen logical design into an executable Microsoft SQL Server database definition. The main database objective was to ensure that the approved 9-table, 3NF schema could be created in SQL Server with all required integrity mechanisms: tables, keys, foreign keys, uniqueness rules, check constraints, default values, indexes, and trigger-based business-rule enforcement.

Unlike the earlier design tasks, this task was not about changing the data model. Its purpose was to faithfully translate the locked schema into physical T-SQL so that later tasks could insert sample data and run reporting queries against a real database instance.

4.5.2 Controlled Agent Generation

The generation was controlled by the `db-design-pipeline:05-generate-ddl` skill and the `generate-ddl -group G05` workflow. The agent first followed the project reading order, including `docs/README.md`, memory files, `docs/templates/README.md`, `docs/design-decisions.md`, and the Task 05 DDL skill. The task-specific skill required the agent to treat `docs/schema-registry.md` as the highest-priority source of truth, with `outputs/04-design-validation-G05.md` and `outputs/03-logical-design-G05.md` used only as supporting references.

The controlled-generation rule was important because Task 05 occurred after schema freeze. The agent was not allowed to invent new tables, change cardinalities, modify foreign-key actions, or edit upstream `outputs/` files directly. Any implementation choice had to match the locked schema registry, SQL Server conventions in `docs/tech-stack.md`, and the recorded design decisions.

4.5.3 Evaluation Criteria

Task 05 was evaluated using the SQL Server DDL file-level checks from the project rubric. Since this task translated the frozen schema into executable SQL, the criteria covered both schema transcription and SQL Server execution correctness.

- **Schema-registry transcription:** every locked table, column, primary key, foreign key, unique constraint, check constraint, default, index, and trigger from `docs/schema-registry.md` must be faithfully implemented.
- **No unauthorized schema drift:** no extra schema objects may be introduced unless explicitly allowed and documented.
- **Nullability and defaults:** nullability rules and default values must match the registry.
- **Trigger scope and business-rule coverage:** trigger behavior must match the approved business-rule enforcement design.

- **T-SQL batch requirements:** the script must start with `SET QUOTED_IDENTIFIER ON` and `GO`; each `CREATE TRIGGER` must be separated by `GO`.
- **Naming and trigger safety:** constraint names must follow `PK_`, `FK_`, `UQ_`, and `CK_` prefixes, and auto-stamp triggers must include recursion guards.
- **Execution evidence:** the evaluation must include a schema-registry transcription audit, naming review, and the latest zero-error DDL compile log.

4.5.4 Output Summary

The task produced a complete SQL Server DDL script implementing the physical schema across nine tables: `departments`, `users`, `spaces`, `facilities`, `space_facilities`, `bookings`, `booking_approvals`, `booking_sessions`, and `maintenance`.

The final script contains 705 lines of T-SQL. It defines the 9 base tables, 12 foreign keys, 12 check constraints, business-key unique constraints, default constraints for statuses, audit timestamps, and soft deletion, 14 supporting indexes, and a filtered unique index, `uq_bookings_active_overlap`. It also defines 21 triggers: 13 business-rule triggers and 8 `updated_at` auto-stamp triggers.

Compilation evidence was stored in `logs/eval/task05/2026-07-02-0140-05-ddl-compile.log`. The verification query confirmed that SQL Server created **9 tables**, **21 triggers**, **12 foreign keys**, and **12 check constraints**.

4.5.5 Database Design Rationale

The DDL directly implements the logical design using four core strategies:

- **Key strategy:** entity tables use surrogate `INT IDENTITY(1,1)` primary keys for stable references, while natural identifiers such as `users.email` and `spaces.space_code` remain protected by `UNIQUE` constraints.
- **Lifecycle normalization:** `bookings`, `booking_approvals`, and `booking_sessions` keep request, decision, and usage-session data separate, with unique `booking_id` foreign keys enforcing 1:0..1 child records.
- **Constraint enforcement:** declarative constraints handle local domains and required values, while triggers and the filtered overlap index enforce cross-row, cross-table, and state-transition rules.
- **Data preservation:** soft deletion is used for `bookings` and `maintenance`, while destructive cascades are limited to dependent approval and session rows.

4.5.6 Issues and Improvements

Task 05 required refinement beyond a simple table transcription. The main improvements focused on SQL Server execution behavior, trigger safety, business-rule traceability, and closing minor validation recommendations from Task 04. Table 10 summarises the major issues found, the corresponding improvements, and their database impact.

Issue Found	Improvement Made	Effect
Expected context files were not recorded in the task trajectory.	The generation workflow was revised to explicitly read and record all required inputs: memory files, schema registry, design decisions, validation report, and logical design artifacts.	Improved process traceability and made the DDL easier to audit against the schema-freeze sources.
<code>CREATE TRIGGER</code> statements are batch-sensitive in SQL Server. If they are not separated correctly, SQL Server can reject the script even when the trigger body is logically correct.	Every trigger definition was placed after a <code>GO</code> separator, and the file starts with <code>SET QUOTED_IDENTIFIER ON</code> so that the filtered unique index can be created safely.	Improved T-SQL portability and allowed the full DDL script to compile cleanly on SQL Server 2019.
Auto-stamp triggers could become unsafe if they repeatedly updated <code>updated_at</code> in response to their own update.	Recursion-protection guards were added to the <code>updated_at</code> triggers across all eight tables that contain an <code>updated_at</code> column and require an auto-stamp trigger.	Preserved the audit trail while preventing accidental recursive update loops.
Business-rule enforcement was present but not easy to trace from DDL alone.	Trigger comments and validation messages were aligned with the corresponding business rules from the schema registry, such as - <code>BR1:</code> , - <code>BR6:</code> , - <code>BR8/BR9:</code> , and - <code>BR19:</code> .	Strengthened traceability from requirements to the physical implementation and made later debugging of failed inserts or updates clearer.
Task 04 recommended optional defensive checks, especially for <code>space_facilities.quantity</code> .	Additional integrity safeguards were added where they did not alter the frozen schema: positive quantity when applied, valid maintenance completion timing, and valid session end timing.	Improved data quality while remaining compatible with the schema-freeze rule because no new tables, relationships, or cardinalities were introduced.

Bảng 10: Issues identified and improvements made during Task 05 iterations

4.5.7 Evaluation Result

Task 05 was evaluated across three iterations using both artifact-level and process-level criteria. The final artifact score was **4.90/5.00**, while the process score reached **4.50/5.00**. The final DDL passed the core implementation checks, including frozen-schema compliance, primary and foreign keys, unique and check constraints, default values, indexes, trigger logic, business-rule traceability, and SQL Server 2019 compilation with zero errors.

The iterative refinement primarily strengthened trigger safety, compile-log persistence, business-rule comments, and additional defensive integrity checks. The key carry-forward lesson was that DDL generation is both a translation task and an execution task: a schema can be conceptually correct but still fail if SQL Server batch rules, filtered-index settings, trigger guards, or compile evidence are handled loosely.

4.6 Task 06: Sample Data Preparation

4.6.1 Database Objective

Task 06 used the frozen Task 05 DDL as the primary source of truth and converted the approved schema into executable SQL Server sample data.

4.6.2 Controlled Agent Generation

The agent used `outputs/05-db-definition-G05.sql` as the locked SQL Server DDL and treated it as the primary implementation reference for sample data generation. The read-only schema context came from `docs/schema-registry.md` and `docs/entity-registry.md`, while the business-rule context came from `outputs/01-business-req-analysis-G05.md` and `req/business-requirement.md`.

4.6.3 Evaluation Criteria

Task 06 was evaluated using the sample-data and proof-case file-level checks from the project rubric. The criteria focused on whether the script populated the frozen schema realistically while proving that the database constraints and triggers behave as intended.

- **FK-safe valid seed data:** sample rows must be realistic and inserted in dependency order.
- **Expected-error proof cases:** each intentional failure must be marked with `- Expected error: ...`, capture the matching SQL Server error, and prove the intended business rule or constraint rather than an unrelated setup failure.
- **Cleanup and rerunnability:** cleanup must use stable Task 06 ownership predicates, and the script must be idempotent or rerunnable against existing Task 06 data.
- **Lifecycle and coverage checks:** lifecycle-relative dates must use one dynamic anchor, and the script must cover role-invalid cases, workflow-produced statuses, table/FK/CHECK/UNIQUE constraints, filtered-unique coverage, and trigger side effects.
- **Verification evidence:** the artifact must provide a static coverage inventory with zero uncovered items, `EXPECTED_ERROR_CASE` and `VERIFY` markers, final execution output under `logs/execution/task06/`, and `rerun/idempotence` evidence when available.

4.6.4 Output Summary

The task produced `outputs/06-sample-data-G05.sql`, a SQL Server sample data script for the Campus Space Management System. The final version contains an assumptions header, `SET QUOTED_IDENTIFIER ON`, an idempotent cleanup-and-reseed section, FK-safe inserts, intentional

expected-error cases, and verification queries.

The final script seeds all nine schema tables: `departments`, `users`, `spaces`, `facilities`, `space_facilities`, `bookings`, `booking_approvals`, `booking_sessions`, and `maintenance`. It covers all required user roles, account statuses, space types, space statuses, booking statuses, booking purposes, and maintenance statuses.

The final seed set includes 9 spaces, 8 Task 06 users, maintenance records, booking approvals, booking sessions, valid booking scenarios, 13 expected-error cases, and 13 verification queries.

4.6.5 Database Design Rationale

Task 06 treated sample data as database validation evidence rather than simple illustrative rows. The script follows the frozen Task 05 DDL without changing the schema, and each seed row or proof case is designed to verify tables, foreign keys, constraints, triggers, and lifecycle rules.

The data is inserted in foreign-key-safe order, from reference and parent tables to dependent booking, approval, session, and maintenance records. The script also uses Task 06-owned natural keys, such as `T06-*` space codes and `t06.*@university.edu` emails, so cleanup-and-reseed can be rerun without removing unrelated data.

Booking lifecycle data follows the normalized workflow implemented by the schema: requests are created in `bookings`, decisions in `booking_approvals`, and check-in/completion evidence in `booking_sessions`. Time-sensitive rows use a single `@now = SYSDATETIME()` anchor so past, current, future, completed, checked-in, and no-show examples remain meaningful across reruns.

Expected-error cases are isolated in transactions and matched against the intended trigger message or constraint name. Together with the final verification queries, they prove overlap prevention, unavailable-space blocking, capacity limits, maintenance conflicts, role checks, duplicate child prevention, soft-delete history, audit timestamps, and maintenance restoration behavior.

4.6.6 Issues and Improvements

Task 06 evolved through several iterations. The main process issue was excessive procedural detail in the command and main skill. That structure encouraged the agent to focus on producing the SQL artifact while treating trajectory recording as a late or easily missed step.

The fix was to use progressive disclosure. The command stayed as a small interface, the main skill stayed as the task router, and phase-specific instructions moved into focused reference files that are read only when needed.

Issue Found	Improvement Made	Effect
Workflow guidance and evidence capture were too scattered: the command and skill held too much procedural detail, and one trajectory treated execution validation as a separate manual step.	Task 06 was reorganized into progressive-disclosure reference files: <code>data-coverage.md</code> , <code>idempotence-and-test-isolation.md</code> , <code>business-rule-proofs.md</code> , <code>sql-style-and-ordering.md</code> , <code>execution-validation.md</code> , and <code>completion-actions.md</code> . The completion reference was strengthened to require planned inputs, final <code>sqlcmd</code> verification, execution-log review, documented fixes, and a final trajectory.	The workflow became easier to audit. Trajectory writing, execution review, and rerun expectations became explicit completion gates, and the final trajectory captured the debugging history with the successful execution log.
The early data script was not reliably rerunnable, and expected-error cases could fail for setup reasons instead of proving the intended constraints.	A cleanup-and-reseed strategy was added using stable T06- space codes, t06. emails, reverse-FK-order deletion, stable natural-key lookups, prerequisite checks, and targeted TRY/CATCH blocks.	The script became rerunnable against a database containing previous Task 06 data, and expected-error cases began proving the intended rules: BR1 overlap, BR2 unavailable spaces, BR3 capacity, BR4 maintenance overlap, BR6/BR7 metadata, BR8/BR9 lifecycle fields, and key declarative constraints.
Final execution still exposed SQL Server runtime issues: <code>QUOTED_IDENTIFIER</code> was effectively off, lookup variables crossed GO batch boundaries, BR8/BR9 tests hit BR4 first, and duplicate <code>space_facilities</code> rows were inserted.	<code>SET QUOTED_IDENTIFIER ON</code> was added, lookup variables were redeclared inside each batch, BR8/BR9 cases were moved to a space without active maintenance, and duplicate facility-link inserts were removed.	Execution became compatible with the Task 05 filtered index. The final log showed all valid sections completing and all 13 expected-error cases catching the intended rule-specific errors.

Bảng 11: Issues identified and improvements made during Task 06 iterations

4.6.7 Evaluation Result

Task 06 was evaluated across four recorded rounds using both artifact-level and process-level criteria. The final artifact score was **5.00/5.00**, while the process score reached **5.00/5.00**. The final script passed the required checks for all nine schema tables, enum/status coverage, FK-safe ordering, cleanup-and-reseed idempotence, expected-error isolation, SQL Server execution, and verification queries.

The iterative refinement primarily strengthened rerunnability, business-rule proofs, expected-error accuracy, and execution-log traceability. The approved sample-data script executed successfully and provided the validated data foundation for Task 07 query design.

4.7 Task 07: Query Design

4.7.1 Database Objective

Task 07 designed executable SQL queries that answer role-specific business questions for the Campus Space Management System. The task produced `outputs/07-query-design-G05.sql`, containing all student queries assembled into a single executable file. Each query follows the 4-field template: business question, target user(s), explanation of usefulness, and a parameterized T-SQL statement using `DECLARE` blocks.

4.7.2 Agent Configuration and Improvement Process

Initial approach. The initial design treated Task 07 as a straightforward generation task: the agent would read the approved schema and sample data, then produce five queries independently. A basic command and skill were written to define the output template and SQL quality rules. While the agent successfully generated syntactically valid queries, the output lacked depth and variety. Queries tended to cluster around simple availability checks and repeated similar patterns across students, resulting in low semantic coverage of the domain's business questions.

Improvement: role-scoped query generation. After reviewing the initial output, the team identified the root cause: the agent had no guidance on which user perspective each student should adopt. Without role constraints, every student's queries converged toward the same generic availability-search pattern.

The command was extended with explicit per-student arguments:

- `-student-name` to assign authorship to each query block
- `-target-users` to restrict each student's queries to a specific role subset (e.g. `facility_manager`, `lecturer`)
- `-num-queries` to control output volume
- `-mode` to support append or overwrite when multiple students contributed to the same file
- `-business-question` to guide the agent to follow each student's intended scenario

The skill was updated with a query distribution table mapping each role to expected query categories: lecturers focused on availability and schedule views; facility staff on approvals, check-in, and maintenance; facility managers on utilization analytics and no-show analysis; department admins on audit trails and inter-department comparisons. This prevented semantic duplication across students and produced a query set with wider operational coverage.

Interaction rule. To protect each student’s contribution, the command required the agent to stop and ask the user before overwriting an existing output file. If `-mode` was not specified and the file already existed, the agent would prompt: *“Output file exists. Append new queries for [student name] to the existing file, or overwrite the entire file?”* Only after an explicit answer would the agent proceed. This prevented silent data loss when multiple students ran the command sequentially.

4.7.3 Per-Student Query Summary

Nguyễn Hữu Phước — Role: Facility Staff.

- **Q1 — Pending approvals for today:** Retrieves all pending booking requests within the next 24 hours so facility staff can review and approve or reject them in a timely manner.
- **Q2 — Real-time occupancy snapshot:** Shows which spaces are currently checked-in or approved, who booked them, and when the session ends, enabling live occupancy monitoring across campus.
- **Q3 — Active maintenance tickets:** Lists all open and in-progress maintenance tickets with problem descriptions, assigned staff, and days open, helping staff prioritise repairs.
- **Q4 — Eligible check-in bookings:** Identifies approved bookings past their start time with no check-in, enabling proactive follow-up on no-shows or late arrivals.
- **Q5 — Session completion report for today:** Retrieves sessions completed on a given date with actual end times, space conditions, and check-in staff, supporting end-of-day space review.

Phạm Hữu Nam — Role: Facility Manager.

- **Q6 — Rejected booking audit trail:** retrieves the full rejection history for a specific lecturer, filtered by their natural key (unique email), including rejection reasons, decision timestamps, and the staff member who processed each decision. Supports investigation of possible bias or procedural issues and accountability reviews.
- **Q7 — Spaces with upcoming bookings but no recent maintenance:** identifies spaces with approved or checked-in bookings scheduled within the next 10 days that have *not* undergone any resolved maintenance or quality inspection within the past month (and are not currently under maintenance), so technicians can be proactively dispatched for pre-use checks on long-neglected, high-risk rooms.
- **Q8 — Department no-show rate analysis:** aggregates confirmed booking and no-show counts per department for the current semester, computes the no-show rate percentage, and ranks departments from worst to best to support enforcement of stricter reservation penalties on departments that over-reserve and under-utilize spaces.
- **Q9 — Cumulative usage hours for competing requesters:** identifies student requesters with overlapping pending or approved bookings for a contested meeting room and time slot, then calculates each requester’s cumulative actual usage hours for the current month from completed booking sessions, enabling fair-access allocation that prioritises under-served (low-usage) users.
- **Q10 — Semester room-type request summary:** aggregates total requests, approval rates, and unique user counts by room category (classroom, laboratory, meeting room, auditorium) for a declared semester window, to support data-driven decisions on space expansion or consolidation.

Cao Quảng Hưng — Role: Student.

- **Q11 — Available spaces with particular facilities:** Finds teaching or study spaces equipped with some particular facilities (such as Projector, WhiteBoard) that are free to book within their desired time window
- **Q12 — Particular events on a date:** Find particular events (such as Seminar or Student Activity) are happening on a given date with specified capacity and organizer.
- **Q13 — Alternative spaces when usual room is blocked:** Learns which room the student uses most from their booking history, checks if it is unavailable then proactively recommends up to a specified number of alternatives on the same floor.
- **Q14 — Lab availability by day of week:** Based on booking history from the past year, retrieves which days of the week have the most available project and computer laboratories
- **Q15 — Department admin's upcoming approved bookings:** A particular department administrators identifies all approved upcoming booking made by members of their department, seeing who booked which space, for what purpose, and when.

Trần Đình Quốc Thắng — Role: Lecturer, Teaching Assistant.

- **Q16 — Lab availability with equipment and conflict checks:** finds computer or project labs for a requested tutorial slot by checking capacity, workstation count, optional projector availability, live room status, confirmed booking overlaps, and unresolved maintenance conflicts.
- **Q17 — Lecturer personal booking timeline:** retrieves a lecturer's semester booking history across all active lifecycle statuses, including room details, decision notes, rejection reasons, approver information, and actual session timestamps.
- **Q18 — Completed TA lab session history:** lists completed computer/project lab sessions for a teaching assistant, including actual duration, condition notes, check-in staff, usage notes, and a summarized facility list.
- **Q19 — Approval lead-time analysis:** aggregates approved and rejected lecturer bookings by purpose, space type, and decision status to show minimum, average, and maximum decision lead time for planning future submissions.
- **Q20 — Upcoming lab readiness alert:** identifies approved upcoming TA lab bookings that may be disrupted by non-available space status or overlapping unresolved maintenance, supporting early backup-room planning.

4.7.4 Issues and Improvements Summary

Table 12 consolidates the significant issues identified across all students during the review cycle, together with the improvements made and their effect on query correctness or maintainability.

4.7.5 Evaluation Result

Task 07 was evaluated on query quality, SQL correctness, and alignment with the 4-field template. Queries that returned zero rows due to sample data coverage gaps were documented with an inline comment noting that the logic was correct and the data seed did not cover the scenario. No query was accepted without at least a syntax verification pass.

Q#	Student	Issue found	Improvement made	Effect
Q6	Nam	Filtered on <code>full_name</code> , which is not unique.	Replaced with <code>DECLARE @requester_id INT;</code> filter now uses the surrogate key <code>b.requester_id</code> .	Eliminates false multi-row matches.
Q9	Nam	<code>COALESCE(actual_end, requested_end)</code> overcounted hours for <code>checked_in</code> sessions still in progress.	Replaced with a CASE expression: <code>completed</code> uses <code>actual_end_time</code> ; <code>checked_in</code> in-progress uses <code>GETDATE()</code> ; month bounds derived via <code>DATEFROMPARTS</code> .	Duration accurate per status; query is self-updating each month.

Bảng 12: Issues and improvements across all Task 07 student queries

5 Conclusion

This project completed an AI-assisted database design pipeline for the Campus Space Management System. Starting from the original business requirements, the workflow produced a requirement analysis, conceptual ERD, logical database design, validation report, SQL Server implementation, sample data script, and business query design. These outputs were not created as isolated files; each stage reused and refined knowledge from earlier stages through the entity registry, schema registry, design decisions, memory files, and evaluation logs. This helped maintain consistency from the initial business problem to the final SQL artifacts.

The final database design supports the main operational needs of campus space management, including user and department management, space and facility tracking, booking requests, approval decisions, check-in sessions, maintenance records, soft deletion, auditing, and reporting. The schema was refined into a 9-table relational design with separated booking responsibilities: `bookings` stores booking requests, `booking_approvals` stores approval and rejection decisions, and `booking_sessions` stores actual usage information such as check-in time, completion time, and room condition. This separation improves normalization, traceability, and database-level rule enforcement.

The project also shows that AI agents become more reliable when their working context is organized explicitly. Global rules in `AGENTS.md`, stable project knowledge in `docs/`, task procedures in `.opencode/commands/` and `.opencode/skills/`, persistent state in `memory/`, and evidence in `logs/` formed a structured workflow rather than a one-time prompting process. This organization reduced context drift, made task execution easier to audit, and allowed corrections from one stage to influence later stages.

Tài liệu

- [1] Our CS486-Project Repository. Link: github.com/AmnO-O/CS486-Project
- [2] Trần Văn Dinh. Build app với AI agent: cách mình tổ chức một dự án để AI làm việc đúng convention ngay từ prompt đầu. Viblo link: viblo.asia/p/build-app-voi-ai-agent
- [3] DeepEval Documentation. AI Agent Evaluation. Link: deepeval.com/guides/guides-ai-agent-evaluation
- [4] Hugo Selbie. A Methodical Approach to Agent Evaluation: Building a Robust Quality Gate. Google Cloud Blog. Link: cloud.google.com/.../a-methodical-approach-to-agent-evaluation

- [5] Braintrust. What is agent evaluation? Link: braintrust.dev/articles/agent-evaluation
- [6] Openlayer. Agent evaluation complete guide: testing AI agents. Link: openlayer.com/blog/post/agent-evaluation-complete-guide-testing-ai-agents
- [7] Daily Dose of Data Science. Six Key Metrics for AI Agent Evaluation. Link: blog.dailydoseofds.com/p/six-key-metrics-for-ai-agent-evaluation
- [8] Kore.ai. What is Agent Traceability? - AI Glossary. Link: kore.ai/ai-glossary/what-is-agent-traceability
- [9] Claude-Mem Documentation. Progressive Disclosure: Claude-Mem's Context Priming Philosophy. Link: docs.claude-mem.ai/progressive-disclosure
- [10] Jekyll Documentation. Front Matter. Link: jekyllrb.com/docs/front-matter
- [11] OpenCode Documentation. Commands. Link: opencode.ai/docs/commands
- [12] OpenCode Documentation. Agent Skills. Link: opencode.ai/docs/skills
- [13] Orlena C. Z. Gotel and Anthony C. W. Finkelstein. An Analysis of the Requirements Traceability Problem. Proceedings of the First IEEE International Conference on Requirements Engineering, 1994. Link: doi.org/10.1109/ICRE.1994.292398